

Foundations of Language Modeling

Chapter 5: Self-Attention and Multi-Head Attention

1 The Value Vector

In the previous chapter, we established that tokens emit a **Query** ($\mathbf{q}$) and a **Key** ($\mathbf{k}$). The dot product $\mathbf{q} \cdot \mathbf{k}$ dictates the *affinity* (or attention weight) between two tokens. However, we do not directly aggregate the raw token embeddings $\mathbf{x}$. Instead, tokens project a third vector: the **Value**.

Definition 1.1 (The Value Vector). *Let $\mathbf{x}_t \in \mathbb{R}^C$ be the embedding of the token at step t , and let d_k be the dimension of the attention head. The token emits a **Value** vector $\mathbf{v}_t \in \mathbb{R}^{d_k}$ via a learned linear projection:*

$$\mathbf{v}_t = \mathbf{W}_V \mathbf{x}_t \quad (1)$$

where $\mathbf{W}_V \in \mathbb{R}^{d_k \times C}$ is the value weight matrix.

Intuition: You can think of $\mathbf{x}$ as the private, internal representation of the token. The Query $\mathbf{q}$ is what the token is looking for, the Key $\mathbf{k}$ is what the token advertises it has, and the Value $\mathbf{v}$ is the actual information the token communicates to others if it is selected by their query.

2 Scaled Dot-Product Attention

A critical numerical issue arises when calculating the affinities $\mathbf{q}_t \cdot \mathbf{k}_i$. If the dimensionality d_k of the head is large, the dot products can grow substantially in magnitude. When these large values are fed into the Softmax function, the probability distribution collapses into a one-hot vector (peaking problem), effectively destroying the gradients necessary for backpropagation.

To counteract this, the dot product is scaled by the inverse square root of the head dimension, $\frac{1}{\sqrt{d_k}}$.

Definition 2.1 (Scaled Dot-Product Attention). *Let $\mathbf{Q} \in \mathbb{R}^{T \times d_k}$, $\mathbf{K} \in \mathbb{R}^{T \times d_k}$, and $\mathbf{V} \in \mathbb{R}^{T \times d_k}$ be the matrices containing the queries, keys, and values for all T tokens in a sequence. Scaled dot-product attention is defined as:*

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V} \quad (2)$$

Note: For an autoregressive decoder (like GPT), a lower-triangular causal mask is applied to $\mathbf{Q}\mathbf{K}^\top$ prior to the Softmax.

Remark 2.2 (Self-Attention vs. Cross-Attention). *In **Self-Attention**, $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are all produced by linear projections of the same input sequence $\mathbf{X}$. In **Cross-Attention** (used in Encoder-Decoder architectures), $\mathbf{Q}$ is produced from one sequence (e.g., the decoder's current state), while $\mathbf{K}$ and $\mathbf{V}$ are produced from a separate sequence (e.g., the encoder's output).*

3 Multi-Head Attention

A single attention head limits the network’s ability to focus on different aspects of the context simultaneously. For example, a token might want to find verbs with one head, adjectives with a second head, and rhyming words with a third.

Definition 3.1 (Multi-Head Attention). *Multi-Head Attention applies h independent attention mechanisms (heads) in parallel. The outputs of these heads are concatenated and linearly projected back to the original dimension C :*

$$\text{MultiHead}(\mathbf{X}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (3)$$

where $\text{head}_i = \text{Attention}(\mathbf{X}\mathbf{W}_Q^{(i)}, \mathbf{X}\mathbf{W}_K^{(i)}, \mathbf{X}\mathbf{W}_V^{(i)})$, and $\mathbf{W}^O \in \mathbb{R}^{C \times C}$ is the output projection matrix. Typically, to keep computational costs constant, the head size is chosen as $d_k = C/h$.

4 Worked Examples and Solved Exercises

4.1 Mathematical Exercise: Variance of the Dot Product

Problem: Let $\mathbf{q}$ and $\mathbf{k}$ be two random vectors in $\mathbb{R}^{d_k}$. Assume their components q_j and k_j are independent and identically distributed (i.i.d.) random variables with mean 0 and variance 1.

1. Prove that the variance of the dot product $\mathbf{q} \cdot \mathbf{k}$ is d_k .
2. Prove that scaling the dot product by $\frac{1}{\sqrt{d_k}}$ restores the variance to 1.

Solution: Part 1: The dot product is given by $S = \sum_{j=1}^{d_k} q_j k_j$. We first find the mean and variance of a single term $q_j k_j$. Because q_j and k_j are independent:

$$\mathbb{E}[q_j k_j] = \mathbb{E}[q_j] \mathbb{E}[k_j] = 0 \cdot 0 = 0 \quad (4)$$

The variance of $q_j k_j$ is given by $\mathbb{E}[(q_j k_j)^2] - (\mathbb{E}[q_j k_j])^2$.

$$\text{Var}(q_j k_j) = \mathbb{E}[q_j^2 k_j^2] - 0 = \mathbb{E}[q_j^2] \mathbb{E}[k_j^2] = \text{Var}(q_j) \text{Var}(k_j) = 1 \cdot 1 = 1 \quad (5)$$

Because the sum consists of d_k independent terms, the variance of the sum is the sum of the variances:

$$\text{Var}(S) = \text{Var}\left(\sum_{j=1}^{d_k} q_j k_j\right) = \sum_{j=1}^{d_k} \text{Var}(q_j k_j) = d_k \cdot 1 = d_k \quad (6)$$

Part 2: Recall the property of variance: $\text{Var}(cX) = c^2 \text{Var}(X)$ for any constant c . Let our scaled dot product be $S_{\text{scaled}} = \frac{1}{\sqrt{d_k}} S$.

$$\text{Var}(S_{\text{scaled}}) = \text{Var}\left(\frac{1}{\sqrt{d_k}} S\right) = \left(\frac{1}{\sqrt{d_k}}\right)^2 \text{Var}(S) = \frac{1}{d_k} \cdot d_k = 1 \quad (7)$$

Conclusion: Scaling by $\frac{1}{\sqrt{d_k}}$ ensures the inputs to the Softmax function maintain unit variance, preventing the gradients from vanishing during initialization.

4.2 Coding Example: A Single Self-Attention Head

Problem: Implement a Head module in PyTorch that performs masked, scaled dot-product self-attention.

Solution:

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class Head(nn.Module):
6     """ One head of self-attention """
7     def __init__(self, n_embd, head_size, block_size):
8         super().__init__()
9         # Linear projections for Query, Key, and Value
10        self.key = nn.Linear(n_embd, head_size, bias=False)
11        self.query = nn.Linear(n_embd, head_size, bias=False)
12        self.value = nn.Linear(n_embd, head_size, bias=False)
13
14        # 'tril' is a buffer, not a learned parameter
```

```

15     self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)
16 ))
17
18     def forward(self, x):
19         B, T, C = x.shape # C is n_embd
20
21         k = self.key(x)    # (B, T, head_size)
22         q = self.query(x)  # (B, T, head_size)
23
24         # Compute attention scores ("affinities")
25         # (B, T, head_size) @ (B, head_size, T) -> (B, T, T)
26         # We scale by 1 / sqrt(head_size) to control variance
27         wei = q @ k.transpose(-2, -1) * (k.shape[-1] ** -0.5)
28
29         # Apply causal mask (decoder block)
30         wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))
31         wei = F.softmax(wei, dim=-1) # (B, T, T)
32
33         # Perform the weighted aggregation of values
34         v = self.value(x) # (B, T, head_size)
35         out = wei @ v      # (B, T, T) @ (B, T, head_size) -> (B, T, head_size)
36         return out

```

4.3 Coding Exercise: Implementing Multi-Head Attention

Problem: Using the Head module defined above, implement a MultiHeadAttention module. It should instantiate h heads in parallel, concatenate their outputs, and pass the result through a final linear projection.

Solution:

```

1     class MultiHeadAttention(nn.Module):
2         """ Multiple heads of self-attention in parallel """
3         def __init__(self, num_heads, head_size, n_embd, block_size):
4             super().__init__()
5             # Instantiate a list of heads using nn.ModuleList
6             self.heads = nn.ModuleList([
7                 Head(n_embd, head_size, block_size) for _ in range(num_heads)
8             ])
9
10            # Final linear projection ( $W^0$ ) back to the original embedding dimension
11            self.proj = nn.Linear(n_embd, n_embd)
12
13            def forward(self, x):
14                # x shape: (B, T, n_embd)
15
16                # Run all heads in parallel and store outputs in a list
17                # Each head returns (B, T, head_size)
18                head_outputs = [h(x) for h in self.heads]
19
20                # Concatenate along the channel dimension (dim=-1)
21                # Since head_size = n_embd / num_heads, the concatenated
22                # output will have shape (B, T, n_embd)
23                out = torch.cat(head_outputs, dim=-1)
24
25                # Apply the output projection
26                out = self.proj(out)
27

```

28
29

```
return out
```